

StarUI Architecture

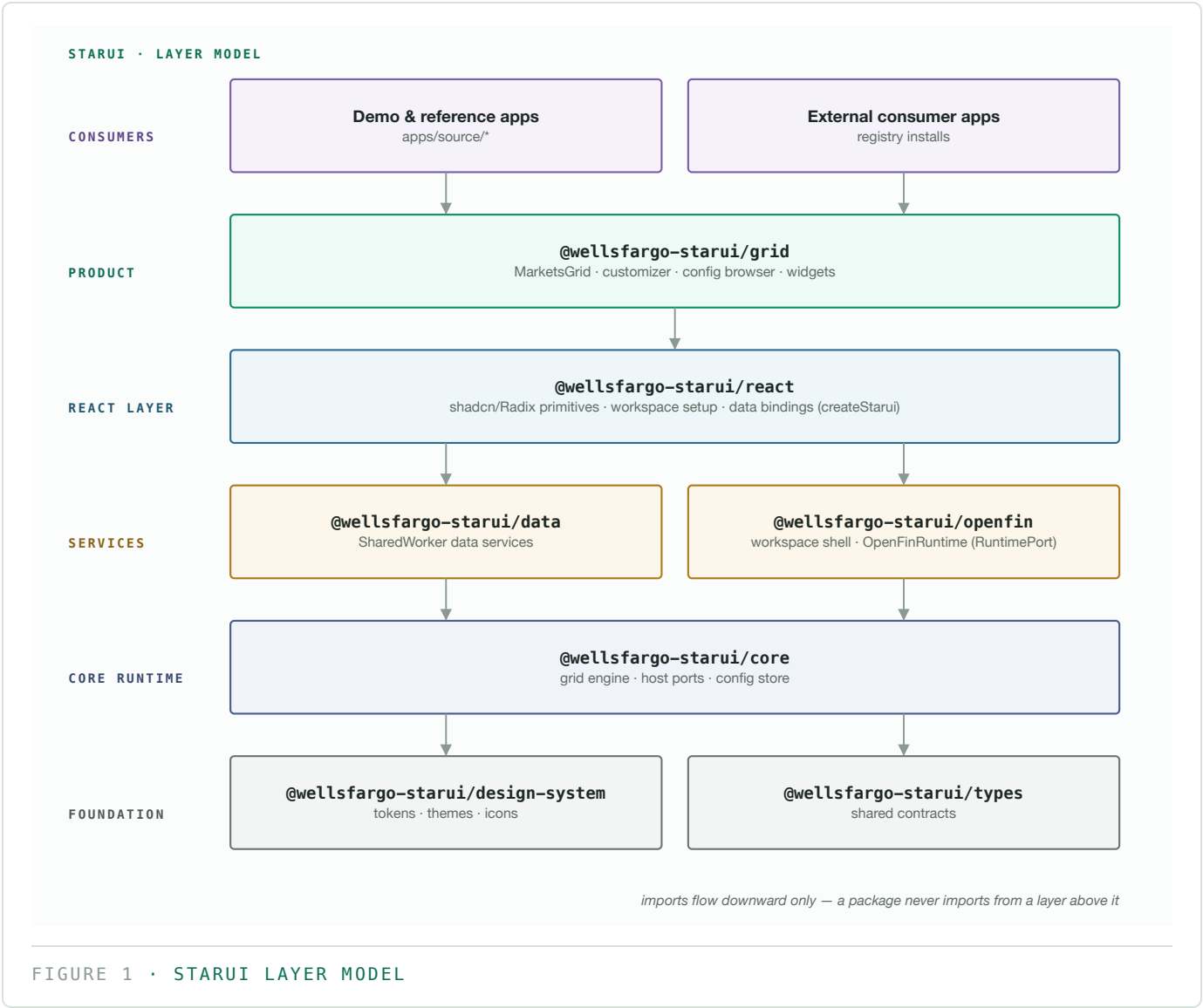
Architecture · docs/latest · generated 2026-08-15

StarUI (the MarketsUI platform) is a layered TypeScript monorepo that ships **seven npm packages** — a design system, a vanilla-TS core runtime, data services, an OpenFin workspace shell, a React component layer, a foundation types package, and the flagship **MarketsGrid** product surface. Consumer applications sit on top and are never imported by the packages.

This document is the canonical architecture reference. For per-package export details see [Package Reference](#) →; for hands-on setup see [Getting Started](#) →.

1. Layer model

Packages are organized into strict layers. **Imports only flow downward** — a package may import from its own layer or below, never above.

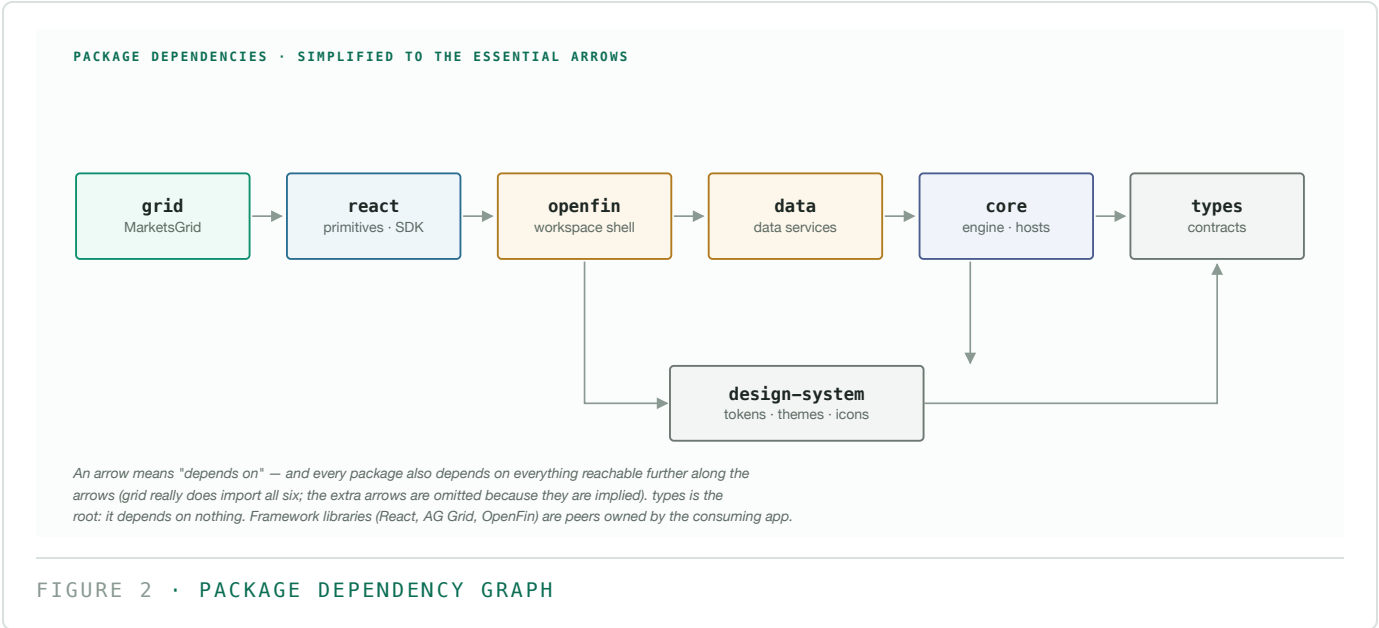


Import boundary rules

RULE	ENFORCED BY
Foundation packages (<code>types</code> , <code>design-system</code>) import nothing except each other	convention + review
<code>core</code> never imports framework adapters (<code>grid</code> , <code>react</code>)	convention + review
Only <code>openfin</code> may import <code>@openfin/core</code>	<code>eslint.config.mjs</code> boundary rules
Apps import packages — never the reverse	package layering (apps are not workspaces)
No package imports from <code>apps/</code>	<code>apps/</code> sits outside the workspace graph

2. Package dependency graph

The dependency structure is a near-chain, so the diagram shows the **essential arrows only**: an arrow means "depends on", and every package also depends on everything reachable further along the arrows (`grid` really does import all six — the extra arrows are omitted because they are implied). Framework libraries such as React, AG Grid and OpenFin are peer dependencies, deliberately owned by the consuming application.



`types` is the root of the graph — it depends on nothing. `grid` is the leaf — it composes everything below into the product surface. Each package's full direct dependency list lives in its `package.json` and in [Package Reference](#) →.

Framework peers (owned by the app)

PEER	REQUIRED BY
<code>react</code> , <code>react-dom</code>	<code>grid</code> , <code>react</code> , <code>design-system</code>
<code>ag-grid-community</code> / <code>ag-grid-enterprise</code> / <code>ag-grid-react</code>	<code>grid</code> (community peer also on <code>core</code> , <code>design-system</code>)
<code>@openfin/core</code> , <code>@openfin/workspace</code> , <code>@openfin/workspace-platform</code>	<code>openfin</code> only
<code>@tanstack/react-query</code>	<code>grid</code> , <code>react</code>
<code>tailwindcss</code>	<code>react</code> , <code>design-system</code>

3. Monorepo layout

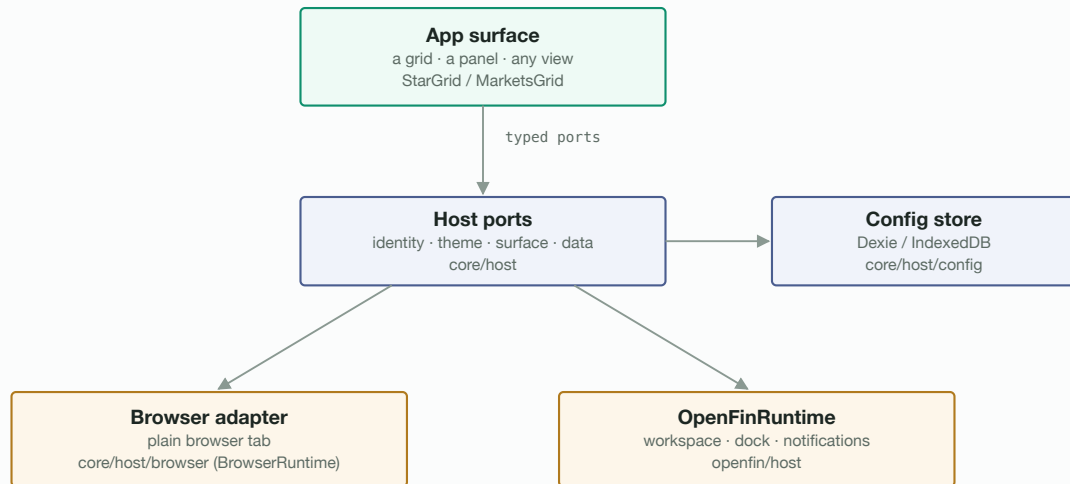
```
stern-bak/
├── packages/
│   ├── types/
│   ├── design-system/
│   ├── core/
│   ├── data/
│   ├── openfin/
│   ├── react-core/
│   └── react-grid/
├── apps/
│   ├── source/
│   ├── tarball/
│   ├── vendor/
│   ├── e2e/ e2e-openfin/
│   └── scripts/
├── scripts/
├── docs/
└── tools/

# @wellsfargo-starui/platform
# the seven buckets - npm workspaces
# @wellsfargo-starui/types
# @wellsfargo-starui/design-system
# @wellsfargo-starui/core
# @wellsfargo-starui/data
# @wellsfargo-starui/openfin
# @wellsfargo-starui/react
# @wellsfargo-starui/grid
# demo apps + e2e - own install root,
# outside workspaces/turbo/coverage
# generated twins (gitignored)
# vendored tarballs (gitignored)
# Playwright suites
# build, packing, coverage, consumer glue
# documentation (latest/ = current set)
# repo-internal checks
```

The build is **Turborepo 2** over npm 10 workspaces; every package's build starts by rimraf-ing its member `dist/` dirs then runs `tsc` per member (plus per-package asset steps — CSS builds, the worker bundle). `npm run build` also regenerates `tsconfig.consumer.json` — the generated `paths` map that lets any consumer typecheck against the repo without being inside it.

4. Runtime architecture — hosts and ports

The core runtime is framework-agnostic vanilla TypeScript. A **host** provides platform capabilities to the app through typed **ports** (`RuntimePort`, `DataPort`); adapters plug the same contracts into different environments.



The same app surface runs unchanged in a browser tab or inside an OpenFin workspace — the adapter provides the same ports.

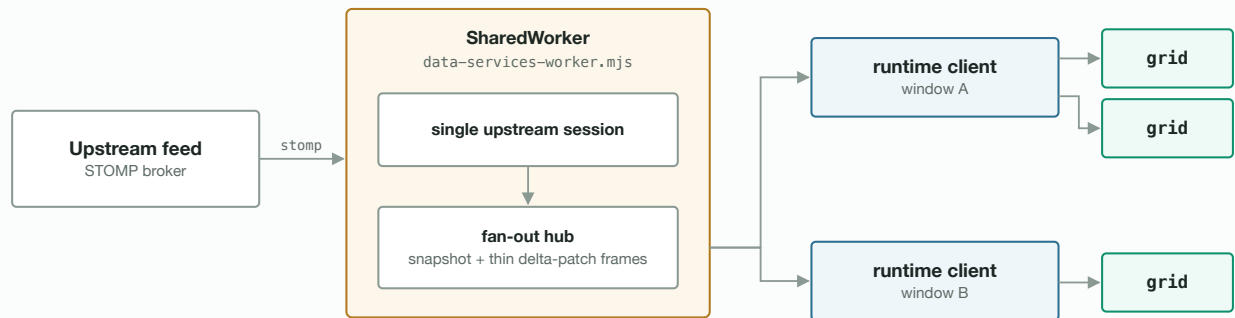
FIGURE 3 · HOST AND PORT RUNTIME MODEL

- **types** defines the port contracts (identity, theme, surface), so every layer agrees on the same shapes.
- **core/host/config** persists configuration through a Dexie (IndexedDB) store; profiles and grid state ride the same mechanism.
- The **browser adapter** (`BrowserRuntime`) runs everything in a plain browser tab; **OpenFinRuntime** (from `@wellsfargo-starui/openfin/host`) provides the same ports inside an OpenFin workspace — the app doesn't change between the two.

5. Data services

`@wellsfargo-starui/data` centralizes market-data distribution in a **SharedWorker**, so N grids across M windows share one upstream connection.

DATA SERVICES · ONE CONNECTION, EVERY WINDOW



N grids across M windows share one upstream connection. Post-snapshot frames carry per-row field patches (thin deltas), not whole rows — see docs/hub-fanout-optimizations.md for the wire format.

FIGURE 4 · DATA SERVICES FLOW

Key properties:

- **One connection, many consumers** — the worker owns the STOMP session; windows attach through `data/runtime/client`.
- **Thin deltas** — post-snapshot live frames broadcast per-row field patches rather than whole rows (see `hub-fanout-optimizations` for the wire format and measured wins).
- **Build-generated asset** — the worker bundle ships as `data/assets/data-services-worker.mjs` and is emitted by `npm run build`; the shared Vite config self-heals it if missing.

6. Build & consumption tracks

The same packages are consumed three ways. The **tarball track exists to keep external consumption honest** — it must work with no aliases and no access to the repo's source tree.

BUILD & CONSUMPTION · THREE TRACKS, ONE TRUTH

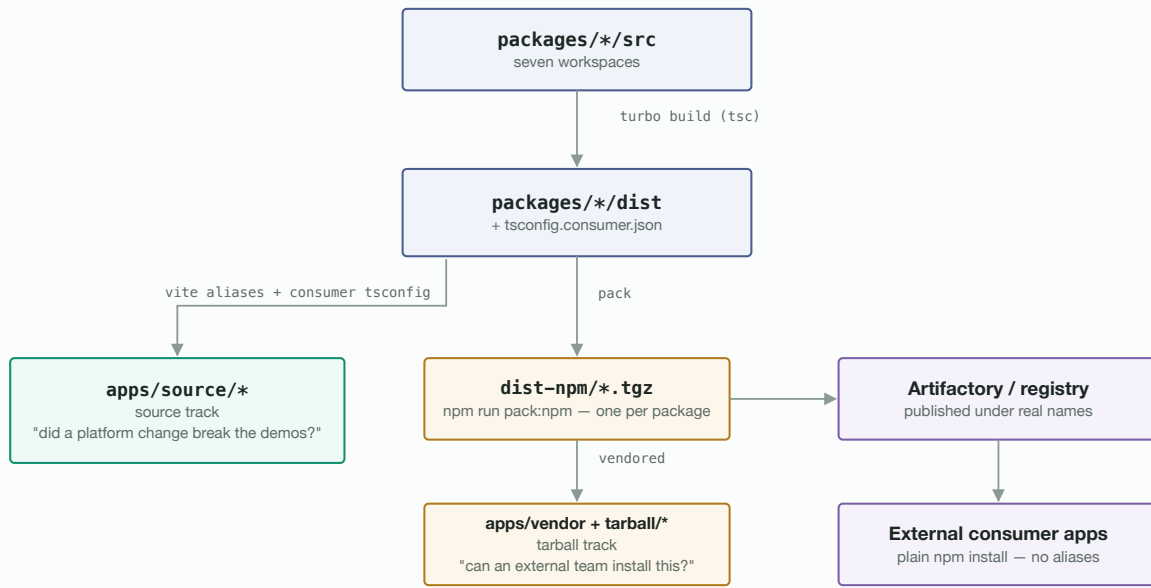


FIGURE 5 · BUILD AND CONSUMPTION TRACKS

TRACK	RESOLUTION	QUESTION IT ANSWERS
source	Vite aliases + generated <code>tsconfig.consumer.json</code>	did a platform change break the demos?
tarball	plain <code>node_modules</code> from vendored tarballs	can an external team install this?
external	registry install, real package names	production consumption

Details and the exact commands live in [Getting Started](#) → and APPS_REPO.md.

7. Quality gates

GATE	COMMAND	SCOPE
Typecheck	<code>npm run typecheck</code>	all seven packages
Unit tests	<code>npm test</code>	Vitest 4 + jsdom across <code>packages/</code>
Coverage	<code>npm run test:coverage</code> + <code>npm run check:coverage</code>	70% per file on lines, statements, functions, branches — <code>all: true</code> , so untested files count
Lint + boundaries	<code>npm run lint:all</code>	ESLint, dependency cycles, design-system token rules, RTL enforcement
External-consumer proof	<code>npm run verify:external</code>	installs the packed tarballs with <code>packages/</code> hidden

GATE	COMMAND	SCOPE
E2E	Playwright under <code>apps/e2e</code>	drives the demo apps

`apps/` is deliberately **outside** every package gate: it is its own npm install root, excluded from workspaces, Turbo, lint, coverage and Sonar — demo apps never dilute the platform's coverage bar.